UNIVERSIDAD RAFAEL LANDÍVAR
FACULTAD DE INGENIERÍA
LENGUAJES FORMALES Y AUTÓMATAS
ING. ROJAS ARGUETA CÉSAR SALVADOR

MANUAL TÉCNICO

Proyecto 2 — Motor de conversión de AFND a AFD y simulación (Fases 1 y 2)

INTEGRANTES:
RODRIGO RUIZ 1037623
JAROD BOLAÑOS 1251621

CIUDAD DE GUATEMALA

Índice

1. Descripción del sistema 4
2. Requisitos y ejecución 5
2.1 Estructura de carpetas 5
3. Arquitectura del software 6
3.1 Dependencias entre módulos 6
4. Modelado de datos 7
4.1 Quíntupla del AFD 7
4.2 Quíntupla del AFND 7
4.3 Función de transición sin condicionales 7
4.4 Estructuras auxiliares 7
4.5 Variables de sesión (main.main) 8
5. Formato de archivos y expresiones regulares 9
5.1 Archivo de AFD 9
5.2 Archivo de AFND 9
5.3 Archivo de cadenas 9
5.4 Expresiones regulares implementadas 9
5.5 Símbolos de expresiones regulares utilizados 10
6. Validaciones 11
6.1 Durante la carga 11
6.2 Motor de validación del AFD 11
6.3 Validación del AFND 11
6.4 Autómatas con errores de carga 11
6.5 Análisis estructural 11
7. Conversión de AFND a AFD (construcción de subconjuntos) 12
7.1 Algoritmo 12
7.2 Reglas aplicadas 12
7.3 Implementación 12
7.4 Verificación del AFD generado 13
7.5 Ejemplo: afnd_dos_destinos.txt 13
8. Simulación 14
8.1 Evaluación sobre el AFD 14
8.2 Simulación directa del AFND y comparación 14
8.3 Evaluación por lotes e historial 14
9. Manejo de errores y resiliencia 15
10. Funcionamiento por opción del menú 16

Opción 1. Crear un AFD manualmente....16
Opción 2. Cargar un AFD desde archivo....16
Opción 3. Crear un AFND manualmente....17
Opción 4. Cargar un AFND desde archivo....17
Opción 5. Mostrar la definición formal y la tabla....17
Opción 6. Validar la estructura del autómata....18
Opción 7. Convertir el AFND en un AFD equivalente....18
Opción 8. Tabla de equivalencias de macroestados....19
Opción 9. Mostrar el AFD generado....19
Opción 10. Evaluar una cadena....19
Opción 11. Evaluar un archivo de cadenas....20
Opción 12. Consultar el historial....20
Opción 13. Análisis estructural....20
Opción 14. Cargar o crear otro autómata....21
Opción 15. Salir....21
11. Diagramas de flujo....22
11.1 Diagrama general....23
11.2 Opción 1....24
11.3 Opción 2....25
11.4 Opción 3....26
11.5 Opción 4....27
11.6 Opción 5....28
11.7 Opción 6 (AFD)....29
11.8 Opción 6 (AFND)....30
11.9 Opción 7....31
11.10 Opción 8....32
11.11 Opción 9....32
11.12 Opción 10....33
11.13 Opción 11....34
11.14 Opción 12....35
11.15 Opción 13....36
11.16 Opción 14....37
11.17 Opción 15....37
12. Casos de prueba....38

1. Descripción del sistema

El sistema es una aplicación de consola escrita en Python 3 para el modelado, la validación, la conversión y la simulación de autómatas finitos. Evoluciona el programa de la Fase 1 sin eliminar ninguna de sus funciones: permite crear o cargar Autómatas Finitos Deterministas (AFD) y, desde la Fase 2, también Autómatas Finitos No Deterministas (AFND), tanto de forma interactiva como desde archivos de texto analizados con expresiones regulares.

A partir de un AFND validado, el sistema aplica el algoritmo de construcción de subconjuntos para obtener un AFD equivalente, muestra la tabla de equivalencias entre macroestados y subconjuntos del AFND, y habilita sobre el resultado todas las herramientas de la Fase 1: definición formal, tabla de transición, validación determinista, análisis estructural, evaluación de cadenas con traza, evaluación por lotes e historial. Cada cadena evaluada sobre el AFD generado se simula también directamente sobre el AFND original para comprobar que ambos reconocen el mismo lenguaje.

Todo el proyecto usa únicamente estructuras nativas de Python (set, frozenset, dict, list, deque y dataclasses); no se utiliza ninguna biblioteca que implemente autómatas ni la conversión.

2. Requisitos y ejecución

Elemento	Detalle
Lenguaje	Python 3.10 o superior
Módulos estándar	sys, os, re, dataclasses, collections (deque). No requiere instalar paquetes con pip.
Sistemas operativos	Windows 10/11, GNU/Linux y macOS
Codificación	main.py reconfigura sys.stdin, sys.stdout y sys.stderr a UTF-8 para leer y mostrar δ, λ, ε y ∅ sin errores en consolas de Windows.
Ejecución	Desde la carpeta `codigo`: `python main.py`

2.1 Estructura de carpetas

```
Proyecto 2/
  codigo/          main.py, automata.py, cargador.py, cargador_afnd.py, validador.py,
                   validador_afnd.py, conversor.py, simulador.py, colores.py
  ejemplos/        archivos .txt de AFD, AFND y cadenas para los casos de prueba
  Documentacion/   documento de análisis y diseño, manuales y diagramas (draw.io)
```

3. Arquitectura del software

El sistema sigue el principio de separación de responsabilidades que pide el enunciado: carga, análisis sintáctico, validación, conversión y simulación viven en módulos distintos, y main.py solo orquesta.

Módulo	Responsabilidad	Funciones o clases principales
main.py	Punto de entrada: menú de 15 opciones, variables de sesión, reglas de acceso entre opciones e impresión de resultados.	main, obtener_afd_operable, accion_* , verificar_afd_generado, evaluar_y_mostrar, Historial
automata.py	Estructuras de datos: quíntupla del AFD y del AFND.	AFD, AFND
cargador.py	Creación manual y lectura de archivos de AFD con expresiones regulares (Fase 1).	crear_afd_manual, cargar_afd_desde_archivo, normalizar_ruta_archivo
cargador_afnd.py	Creación manual y lectura de archivos de AFND (varios destinos y ∅). Reutiliza funciones de cargador.py.	crear_afnd_manual, cargar_afnd_desde_archivo, _separar_destinos
validador.py	Validación del AFD, estado de trampa y análisis estructural (Fase 1). También valida el AFD generado.	validar_estructura, analizar_estructura, completar_con_estado_trampa
validador_afnd.py	Validación de la quíntupla de un AFND.	validar_estructura_afnd
conversor.py	Algoritmo de construcción de subconjuntos.	construir_afd_desde_afnd, formatear_subconjunto
simulador.py	Simulación de cadenas sobre el AFD (traza, lotes) y simulación directa del AFND para comparar.	evaluar_cadena, evaluar_lote, evaluar_cadena_afnd, imprimir_comparacion
colores.py	Colores ANSI para la consola, sin bibliotecas externas.	exito, error, advertencia, info, titulo

3.1 Dependencias entre módulos

```
main.py ──> cargador.py, cargador_afnd.py, validador.py, validador_afnd.py,
            conversor.py, simulador.py, automata.py, colores.py
cargador_afnd.py ──> cargador.py (reutiliza patrones y funciones auxiliares), automata.py
conversor.py ──> automata.py (crea el AFD generado con la misma clase AFD)
simulador.py ──> conversor.py (formatear_subconjunto para la traza del AFND)
```

4. Modelado de datos

4.1 Quíntupla del AFD

La clase AFD (automata.py, líneas 23-77) representa M = (Q, Σ, δ, q0, F). δ es un diccionario cuya clave es la tupla (estado, símbolo) y cuyo valor es un único estado destino.

```
class AFD:
    def __init__(self, nombre, Q, sigma, delta, q0, F):
        self.nombre = nombre      # str
        self.Q = set(Q)           # set[str]
        self.sigma = set(sigma)   # set[str]
        self.delta = dict(delta)  # dict[(str, str), str]
        self.q0 = q0              # str
        self.F = set(F)           # set[str]
        self.validado = False     # True solo tras pasar la validación
        self.errores_carga = []   # errores de lectura del archivo de origen

    def transicion(self, estado, simbolo):
        return self.delta.get((estado, simbolo))   # None = transición no definida
```

4.2 Quíntupla del AFND

La clase AFND (líneas 80-141) tiene la misma quíntupla, pero cada valor de δ es un frozenset de destinos (cero, uno o varios). Un par sin transición declarada se interpreta como ∅, que en un AFND es válido.

```
class AFND:
    def __init__(self, nombre, Q, sigma, delta, q0, F):
        ...
        self.delta = {clave: frozenset(destinos) for clave, destinos in delta.items()}
        ...

    def transicion(self, estado, simbolo):
        return self.delta.get((estado, simbolo), frozenset())   # ∅ si no hay destinos
```

4.3 Función de transición sin condicionales

En ambos casos δ se consulta con `delta.get((estado, simbolo))` en tiempo constante, sin ninguna cadena de if/elif por autómata específico. Por eso el programa funciona con cualquier AFD o AFND que respete el formato, sin modificar el código.

4.4 Estructuras auxiliares

Estructura	Módulo	Campos y uso
Paso	simulador.py	estado_origen, simbolo, estado_destino: un paso de la traza del AFD.
ResultadoEvaluacion	simulador.py	cadena, traza, veredicto, motivo: resultado de evaluar una cadena en el AFD.
PasoAFND	simulador.py	conjunto_origen, simbolo, conjunto_destino: un paso de la simulación directa del AFND.
ResultadoEvaluacionAFND	simulador.py	cadena, traza, veredicto, motivo: resultado de simular el AFND.

Estructura	Módulo	Campos y uso
AnalisisEstructural	validador.py	alcanzables, inaccesibles, finales_alcanzables, lenguaje_vacio.
RegistroHistorial / Historial	main.py	cadena, veredicto, traza, motivo, automata, veredicto_afnd; lista de evaluaciones del autómata activo.
tabla_equivalencias	conversor.py / main.py	dict[etiqueta] -> frozenset: macroestado del AFD y el subconjunto del AFND que representa.

4.5 Variables de sesión (main.main)

Variable	Contenido
automata_cargado	AFD o AFND creado/cargado con las opciones 1 a 4 (None si no hay).
afd_generado	AFD obtenido en la opción 7 (None mientras no se convierta).
tabla_equivalencias	Resultado de la conversión; la usa la opción 8.
historial	Instancia de Historial; se reinicia en las opciones 1 a 4 y 14.

5. Formato de archivos y expresiones regulares

5.1 Archivo de AFD

```
NOMBRE=AFD_AB
ESTADOS=q0,q1,q2
ALFABETO=a,b
INICIAL=q0
FINALES=q2
TRANSICIONES:
q0,a,q1
q0,b,q0
q1,a,q1
q1,b,q2
q2,a,q1
q2,b,q0
```

5.2 Archivo de AFND

Convención documentada y validada: | separa varios destinos; ∅ (o el destino vacío) representa ausencia de destinos; `TIPO=AFND` es opcional pero, si aparece, debe decir AFND. Las transiciones ε no forman parte del alcance de la Fase 2 y se reportan como error.

```
NOMBRE=AFND_DOS_DESTINOS
TIPO=AFND
ESTADOS=q0,q1,q2
ALFABETO=a,b
INICIAL=q0
FINALES=q2
TRANSICIONES:
q0,a,q0|q1
q0,b,q0
q1,a,∅
q1,b,q2
q2,a,q2
q2,b,q2
```

5.3 Archivo de cadenas

Una cadena por línea; una línea vacía representa λ. Se lee completo y cada línea se evalúa con el mismo algoritmo de la opción 10.

5.4 Expresiones regulares implementadas

Patrón	Expresión regular	Uso
PATRON_NOMBRE	^NOMBRE=(.+)$	Nombre del autómata
PATRON_TIPO	^TIPO=(.+)$	Tipo declarado (AFD o AFND)
PATRON_ESTADOS	^ESTADOS=(.+)$	Conjunto Q
PATRON_ALFABETO	^ALFABETO=(.+)$	Alfabeto Σ
PATRON_INICIAL	^INICIAL=(.+)$	Estado inicial q0
PATRON_FINALES	^FINALES=(.*)$	Conjunto F (puede ir vacío)

Patrón	Expresión regular	Uso
PATRON_ENCABEZADO_TRANSICIONES	^TRANSICIONES:$	Inicio de la sección de transiciones
PATRON_TRANSICION (AFD)	^([A-Za-z0-9_]+)\s*,\s*([^\s,]+)\s*,\s*([A-Za-z0-9_]+)$	origen,simbolo,destino
PATRON_TRANSICION_AFND	^([A-Za-z0-9_]+)\s*,\s*([^\s,]+)\s*,\s*(.*)$	origen,simbolo,destinos
PATRON_DESTINOS_AFND	^(?:∅|[A-Za-z0-9_]+(?:\s*|\s*[A-Za-z0-9_]+)*)?$	Columna de destinos: ∅, vacío, q1 o q0|q1|...
PATRON_IDENTIFICADOR	^[A-Za-z0-9_]+$	Nombre de estado válido

5.5 Símbolos de expresiones regulares utilizados

Símbolo	Nombre	Función en el análisis del archivo
^ y $	Anclas de inicio y fin	La coincidencia debe abarcar la línea completa; no se acepta texto adicional.
.	Comodín	Cualquier carácter (por ejemplo, el valor de NOMBRE=).
* y +	Cuantificadores	Cero o más / uno o más repeticiones del elemento anterior.
(...)	Grupo de captura	Extrae la parte que se guarda en variables de Python.
(?: ...)	Grupo sin captura	Agrupa la repetición `|destino` sin crear un grupo adicional.
[...] y [^ ...]	Clase y clase negada	Caracteres permitidos (identificadores) o excluidos (espacios y comas en el símbolo).
\s	Espacio en blanco	Combinado con * tolera espacios alrededor de comas y de |.
|	Barra escapada	Reconoce el separador literal de destinos del AFND.
?	Opcional	Permite la columna de destinos vacía (sin destinos).

6. Validaciones

6.1 Durante la carga

- Manual (opciones 1 y 3): cada dato se valida al ingresarlo y se vuelve a pedir si es inválido: nombre no vacío; Q y Σ sin vacíos ni duplicados; Σ sin ε; q0 ∈ Q; F ⊆ Q; origen y destinos ∈ Q; símbolo ∈ Σ; par (estado, símbolo) sin repetir.
- Archivo (opciones 2 y 4): cada línea debe coincidir con un patrón; los errores se acumulan con su número de línea y la lectura continúa. Se detectan campos declarados dos veces, ε, pares repetidos, destinos mal formados o repetidos y un TIPO que no corresponde a la opción usada.
- Si faltan NOMBRE, ESTADOS, ALFABETO o INICIAL, el autómata no se construye. Si hay otros errores, se construye de forma parcial y los errores quedan en `errores_carga`.

6.2 Motor de validación del AFD

- q0 ∈ Q y F ⊆ Q.
- Consistencia: en cada transición, origen ∈ Q, símbolo ∈ Σ y destino ∈ Q.
- Determinismo y completitud: exactamente una transición por cada par de Q × Σ. Los pares faltantes se listan como transiciones incompletas.
- Estado de trampa: si el usuario lo pide, se agrega `q_trampa` (o `q_trampa_2`, ... si el nombre existe), se dirigen a él las transiciones faltantes y se le agregan auto-transiciones; nunca es final.
- Clasificación: AFD válido y completo, AFD incompleto, definición no determinista (detectada en la carga) o definición inválida.

6.3 Validación del AFND

validar_estructura_afnd verifica q0 ∈ Q, F ⊆ Q y, por cada transición, origen ∈ Q, símbolo ∈ Σ y destinos ⊆ Q (resta de conjuntos). No existe el concepto de AFND incompleto: un par sin transición es ∅.

6.4 Autómatas con errores de carga

Antes de validar, main.reportar_errores_de_carga revisa `errores_carga`. Si no está vacía, muestra esos errores y los errores estructurales de lo que sí se leyó, y deja el autómata como no validado. Así, un archivo con errores sintácticos nunca produce un autómata «válido» ni puede convertirse o evaluarse.

6.5 Análisis estructural

- Alcanzables: recorrido del grafo desde q0 siguiendo δ con cada símbolo de Σ.
- Inaccesibles: Q − alcanzables.
- Finales alcanzables: F ∩ alcanzables.
- Lenguaje potencialmente vacío: si F ∩ alcanzables = ∅.

7. Conversión de AFND a AFD (construcción de subconjuntos)

7.1 Algoritmo

```
macroestado_inicial <- {q0};  etiqueta A
pendientes <- cola con {q0}
mientras pendientes no esté vacía:
    S <- sacar el primero de pendientes
    para cada símbolo a de Σ (en orden):
        S' <- unión de δ_AFND(q, a) para cada q en S
        si S' es nuevo: asignarle la siguiente etiqueta (A, B, ..., Z, AA, ...) y encolarlo
        δ_AFD(etiqueta(S), a) <- etiqueta(S')
F_AFD <- { etiqueta(S) : S ∩ F_AFND ≠ ∅ }
```

7.2 Reglas aplicadas

- El estado inicial del AFD es el macroestado {q0}.
- Cada subconjunto nuevo se registra y se procesa hasta que no quedan macroestados pendientes (recorrido en anchura con `deque`).
- El conjunto vacío ∅ se trata como un macroestado más: si se alcanza, se convierte en el estado de trampa, que vuelve a sí mismo con cada símbolo porque la unión de destinos de un conjunto vacío es ∅.
- Un macroestado es final si contiene al menos un estado final del AFND, aunque también contenga estados no finales.
- El AFD resultante tiene exactamente un destino por cada par (macroestado, símbolo).

7.3 Implementación

conversor.construir_afd_desde_afnd (líneas 54-108):

```
def construir_afd_desde_afnd(afnd):
    macroestado_inicial = frozenset({afnd.q0})

    etiquetas = {macroestado_inicial: _siguiente_etiqueta(0)}
    orden_descubrimiento = [macroestado_inicial]
    pendientes = deque([macroestado_inicial])
    delta_afd = {}

    while pendientes:
        actual = pendientes.popleft()
        for simbolo in sorted(afnd.sigma):
            destino = frozenset()
            for estado in actual:
                destino |= afnd.delta.get((estado, simbolo), frozenset())

            if destino not in etiquetas:
                etiquetas[destino] = _siguiente_etiqueta(len(etiquetas))
                orden_descubrimiento.append(destino)
                pendientes.append(destino)

            delta_afd[(etiquetas[actual], simbolo)] = etiquetas[destino]

    Q_afd = set(etiquetas.values())
    F_afd = {
        etiquetas[subconjunto]
```

```
        for subconjunto in orden_descubrimiento
        if subconjunto & afnd.F
    }

    afd_generado = AFD(
        nombre=f"{afnd.nombre}_AFD",
        Q=Q_afd,
        sigma=afnd.sigma,
        delta=delta_afd,
        q0=etiquetas[macroestado_inicial],
        F=F_afd,
    )

    tabla_equivalencias = {
        etiquetas[subconjunto]: subconjunto for subconjunto in orden_descubrimiento
    }
    return afd_generado, tabla_equivalencias
```

7.4 Verificación del AFD generado

El AFD generado se almacena con la misma clase AFD que un autómata cargado directamente. El conversor no lo marca como válido: main.verificar_afd_generado lo pasa por validador.validar_estructura (el mismo motor de la Fase 1) para comprobar que es determinista y completo, y luego muestra su análisis estructural. Solo entonces se habilitan las opciones 8 a 13.

7.5 Ejemplo: afnd_dos_destinos.txt

Macroestado	Subconjunto	Con a	Con b	¿Final?
A (inicial)	{q0}	{q0, q1} → B	{q0} → A	No
B	{q0, q1}	{q0, q1} → B	{q0, q2} → C	No
C	{q0, q2}	{q0, q1, q2} → D	{q0, q2} → C	Sí
D	{q0, q1, q2}	{q0, q1, q2} → D	{q0, q2} → C	Sí

8. Simulación

8.1 Evaluación sobre el AFD

simulador.evaluar_cadena parte de q0 y aplica δ símbolo por símbolo. Rechaza con motivo si un símbolo no pertenece a Σ o si falta una transición; al terminar, acepta si el estado actual pertenece a F. La cadena vacía se acepta solo si q0 ∈ F. Cada paso se guarda en la traza.

8.2 Simulación directa del AFND y comparación

simulador.evaluar_cadena_afnd (líneas 178-212) sigue todos los caminos del AFND a la vez manteniendo el conjunto de estados activos. Es la misma unión que calcula la construcción de subconjuntos, pero solo para la cadena evaluada; por eso, si la conversión es correcta, el veredicto coincide siempre con el del AFD generado. Las opciones 10 y 11 muestran ambos veredictos e indican si coinciden; la opción 11 resume cuántas cadenas coinciden.

```
def evaluar_cadena_afnd(afnd, cadena):
    activos = frozenset({afnd.q0})
    traza = []

    for simbolo in cadena:
        if simbolo not in afnd.sigma:
            return ResultadoEvaluacionAFND(
                cadena=cadena,
                traza=traza,
                veredicto="Rechazada",
                motivo=f"El símbolo '{simbolo}' no pertenece al alfabeto Sigma =
{sorted(afnd.sigma)}.",
            )
        siguientes = frozenset()
        for estado in activos:
            siguientes |= afnd.transicion(estado, simbolo)
        traza.append(PasoAFND(activos, simbolo, siguientes))
        activos = siguientes

    veredicto = "Aceptada" if activos & afnd.F else "Rechazada"
    return ResultadoEvaluacionAFND(cadena=cadena, traza=traza, veredicto=veredicto, motivo=None)
```

8.3 Evaluación por lotes e historial

La opción 11 lee el archivo de cadenas dentro de un try/except, evalúa cada línea y muestra un resumen (total, aceptadas, rechazadas y, si aplica, coincidencias). Todas las evaluaciones se guardan en el historial con su traza, motivo y veredicto del AFND.

9. Manejo de errores y resiliencia

Situación	Mecanismo	Resultado
Archivo inexistente o ilegible	try/except FileNotFoundError y OSError	Mensaje con la ruta; se vuelve al menú.
Línea con sintaxis inválida	Las expresiones regulares no coinciden; se acumula el error con su número de línea	Se listan todos los errores; el autómata queda parcial y no puede validarse.
Estados o símbolos no declarados	validar_estructura / validar_estructura_afnd	Lista de errores; conversión y evaluación bloqueadas.
Transiciones ε	SIMBOLOS_PROHIBIDOS	Se informa que no forman parte del alcance de la fase.
Opción del menú inválida	pedir_opcion valida isdigit() y el rango 1-15	Mensaje y se vuelve a mostrar el menú.
Opción sin requisitos previos	hay_automata_cargado, obtener_afd_operable, accion_convertir	Mensaje que indica el paso que falta.
Ctrl+C o fin de la entrada	try/except KeyboardInterrupt y EOFError en el punto de entrada	Cierre controlado, sin traceback.
Caracteres δ, λ, ∅ en Windows	Reconfiguración de stdin/stdout/stderr a UTF-8	Se leen y muestran correctamente.

10. Funcionamiento por opción del menú

Opción 1. Crear un AFD manualmente

Acciones

1. Pide el nombre (no vacío).
2. Pide Q y Σ separados por comas; rechaza vacíos, duplicados y ε.
3. Pide q0 hasta que pertenezca a Q y F hasta que sea subconjunto de Q.
4. Pide transiciones `origen,simbolo,destino` hasta `FIN`, rechazando en el acto las inválidas o repetidas.
5. Construye el AFD con validado = False, reinicia afd_generado, tabla_equivalencias e historial.

Datos de entrada

- Nombre, Q, Σ, q0, F (puede ser vacío) y transiciones (str).

Variables

- nombre: str; Q, sigma, F: set[str]; q0: str; delta: dict[(str, str), str]; afd: AFD.

Condiciones y cálculos

- `len(elementos) == len(set(elementos))` para Q, Σ y F.
- q0 ∈ Q; F ⊆ Q; origen, destino ∈ Q; símbolo ∈ Σ; `(origen, simbolo) not in delta`.

Opción 2. Cargar un AFD desde archivo

Acciones

1. Pide y normaliza la ruta (comillas, ~, variables de entorno, separadores).
2. Lee el archivo y reconoce cada línea con los patrones de la sección 5.
3. Acumula los errores con número de línea sin detener la lectura.
4. Construye el AFD si están los componentes obligatorios y guarda los errores en errores_carga.
5. Reinicia los datos de la sesión anterior.

Datos de entrada

- Ruta del archivo .txt.

Variables

- ruta, lineas, errores, numero_linea; nombre, Q, sigma, q0, F, delta; dentro_de_transiciones: bool; transiciones_vistas y campos_vistos: set.

Condiciones y cálculos

- Par repetido o destinos con | → error (sería un AFND).
- TIPO distinto de AFD → error que sugiere la opción 4.
- Faltan NOMBRE/ESTADOS/ALFABETO/INICIAL → no se construye.

Opción 3. Crear un AFND manualmente

Acciones

1. Pide nombre, Q, Σ, q0 y F igual que la opción 1 (reutiliza las funciones de cargador.py).
2. Pide transiciones `origen,simbolo,destino1|destino2|...` (∅ o vacío = sin destinos) hasta `FIN`.
3. Construye el AFND y reinicia los datos de la sesión.

Datos de entrada

- Nombre, Q, Σ, q0, F y transiciones.

Variables

- delta: dict[(str, str), frozenset[str]]; destinos: frozenset; afnd: AFND.

Condiciones y cálculos

- Destinos que cumplan PATRON_DESTINOS_AFND, sin repetidos y todos en Q.
- Cada par (origen, símbolo) se declara una sola vez; los destinos múltiples van juntos con `|`.
- ε prohibido.

Opción 4. Cargar un AFND desde archivo

Acciones

1. Igual que la opción 2, pero cada transición se reconoce con PATRON_TRANSICION_AFND y su columna de destinos con PATRON_DESTINOS_AFND.
2. Construye el AFND y guarda los errores en errores_carga.

Datos de entrada

- Ruta del archivo .txt.

Variables

- Las mismas de la opción 2, con delta: dict[(str, str), frozenset[str]].

Condiciones y cálculos

- TIPO debe ser AFND si aparece.
- Destinos inválidos (`q0|`, `q0||q1`, `q0,q1`) o repetidos → error.
- La pertenencia de los destinos a Q se verifica en la opción 6.

Opción 5. Mostrar la definición formal y la tabla

Acciones

1. Verifica que haya un autómata cargado.
2. Imprime la quíntupla con Q, Σ, F y δ ordenados.
3. Imprime la tabla Q × Σ marcando `->` el inicial y `*` los finales; en un AFND cada celda es un conjunto o ∅.

Datos de entrada

- Ninguna.

Variables

- estados, simbolos: list[str]; ancho_estado, ancho_celda: int; celdas: dict.

Condiciones y cálculos

- Los anchos se calculan con el texto más largo para que la tabla no se desalinee.

Opción 6. Validar la estructura del autómata

Acciones

1. Si hay errores de carga, los muestra junto con los estructurales y deja el autómata sin validar.
2. AFD: valida, ofrece el estado de trampa si está incompleto y, si queda válido, muestra el análisis estructural.
3. AFND: valida la quíntupla; si ya existe un AFD generado, lo vuelve a validar.

Datos de entrada

- Solo para un AFD incompleto: respuesta S/N para agregar el estado de trampa.

Variables

- errores: list[str]; incompletas: list[(str, str)]; respuesta, nombre_trampa: str; analisis: AnalisisEstructural.

Condiciones y cálculos

- Reglas de las secciones 6.2 a 6.4; validado = True solo si no hay errores ni incompletas.

Opción 7. Convertir el AFND en un AFD equivalente

Acciones

1. Verifica que el autómata cargado sea un AFND validado.
2. Ejecuta construir_afd_desde_afnd.
3. Valida el AFD generado con el validador de la Fase 1 y muestra su análisis estructural.
4. Guarda afd_generado y tabla_equivalencias.

Datos de entrada

- Ninguna.

Variables

- etiquetas: dict[frozenset, str]; orden_descubrimiento: list; pendientes: deque; delta_afd: dict; Q_afd, F_afd: set.

Condiciones y cálculos

- Sección 7.

Opción 8. Tabla de equivalencias de macroestados

Acciones

1. Verifica que exista una conversión.
2. Imprime cada etiqueta, su subconjunto del AFND ({q0, q1} o ∅) y si es inicial, final o estado de trampa.

Datos de entrada

- Ninguna.

Variables

- tabla_equivalencias: dict[str, frozenset]; afd_generado: AFD.

Condiciones y cálculos

- Final si `etiqueta in afd_generado.F`; trampa si el subconjunto está vacío.

Opción 9. Mostrar el AFD generado

Acciones

1. Verifica que exista una conversión.
2. Imprime la definición formal y la tabla de transición del AFD generado con las mismas funciones de la opción 5.

Datos de entrada

- Ninguna.

Variables

- afd_generado: AFD.

Condiciones y cálculos

- El AFD generado es completo: ninguna celda muestra `-`.

Opción 10. Evaluar una cadena

Acciones

1. Obtiene el AFD operable: el AFD validado o, si se cargó un AFND, el AFD generado.
2. Pide la cadena (vacía = λ), la evalúa e imprime la traza y el veredicto.
3. Si el origen es un AFND, lo simula directamente, imprime su traza de conjuntos y compara los veredictos.
4. Guarda el resultado en el historial.

Datos de entrada

- Cadena (str); puede ser vacía o contener símbolos fuera de Σ.

Variables

- resultado: ResultadoEvaluacion; resultado_afnd: ResultadoEvaluacionAFND; coinciden: bool.

Condiciones y cálculos

- Sección 8.
- No se evalúa un AFND sin convertir.

Opción 11. Evaluar un archivo de cadenas

Acciones

1. Pide la ruta y lee las cadenas (una por línea).
2. Evalúa cada una como en la opción 10.
3. Imprime el resumen: total, aceptadas, rechazadas y, si aplica, coincidencias entre AFND y AFD.

Datos de entrada

- Ruta del archivo de cadenas.

Variables

- cadenas: list[str]; resumen: dict; coincidencias: int.

Condiciones y cálculos

- Lectura protegida con try/except; mismas reglas que la opción 10.

Opción 12. Consultar el historial

Acciones

1. Si no hay evaluaciones, lo informa.
2. Si las hay, muestra cada una con su autómata, cadena, veredicto, traza, motivo y veredicto del AFND si aplica.

Datos de entrada

- Ninguna.

Variables

- historial: Historial con _registros: list[RegistroHistorial].

Condiciones y cálculos

- El historial pertenece al autómata activo y se reinicia en las opciones 1 a 4 y 14.

Opción 13. Análisis estructural

Acciones

1. Obtiene el AFD operable.
2. Muestra alcanzables, inaccesibles, finales alcanzables y si el lenguaje podría ser vacío.

Datos de entrada

- Ninguna.

Variables

- analisis: AnalisisEstructural; pendientes: list (pila del recorrido).

Condiciones y cálculos

- Sección 6.5.

Opción 14. Cargar o crear otro autómata

Acciones

1. Descarta automata_cargado, afd_generado, tabla_equivalencias e historial.
2. Muestra un submenú (0-4) para crear o cargar el siguiente autómata sin reiniciar el programa; 0 vuelve al menú.

Datos de entrada

- Opción del submenú (0-4).

Variables

- acciones: dict[str, función].

Condiciones y cálculos

- Se repite la pregunta hasta recibir una opción válida.

Opción 15. Salir

Acciones

1. Imprime un mensaje de despedida y termina el bucle principal.

Datos de entrada

- Ninguna.

Variables

- Ninguna.

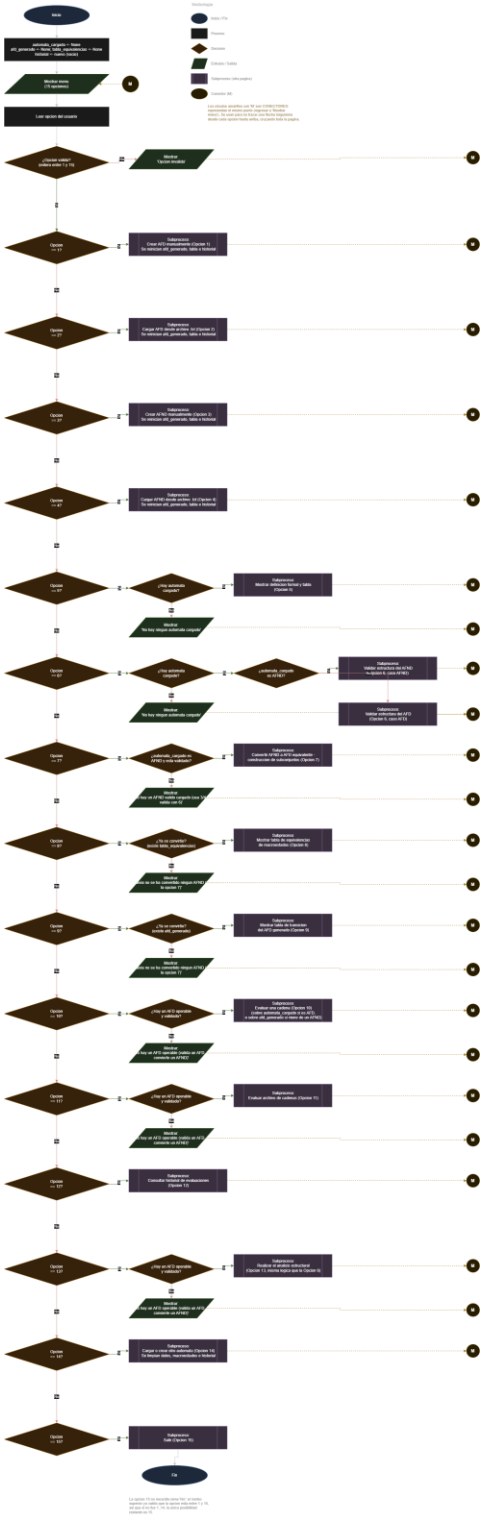
Condiciones y cálculos

- Ninguna.

11. Diagramas de flujo

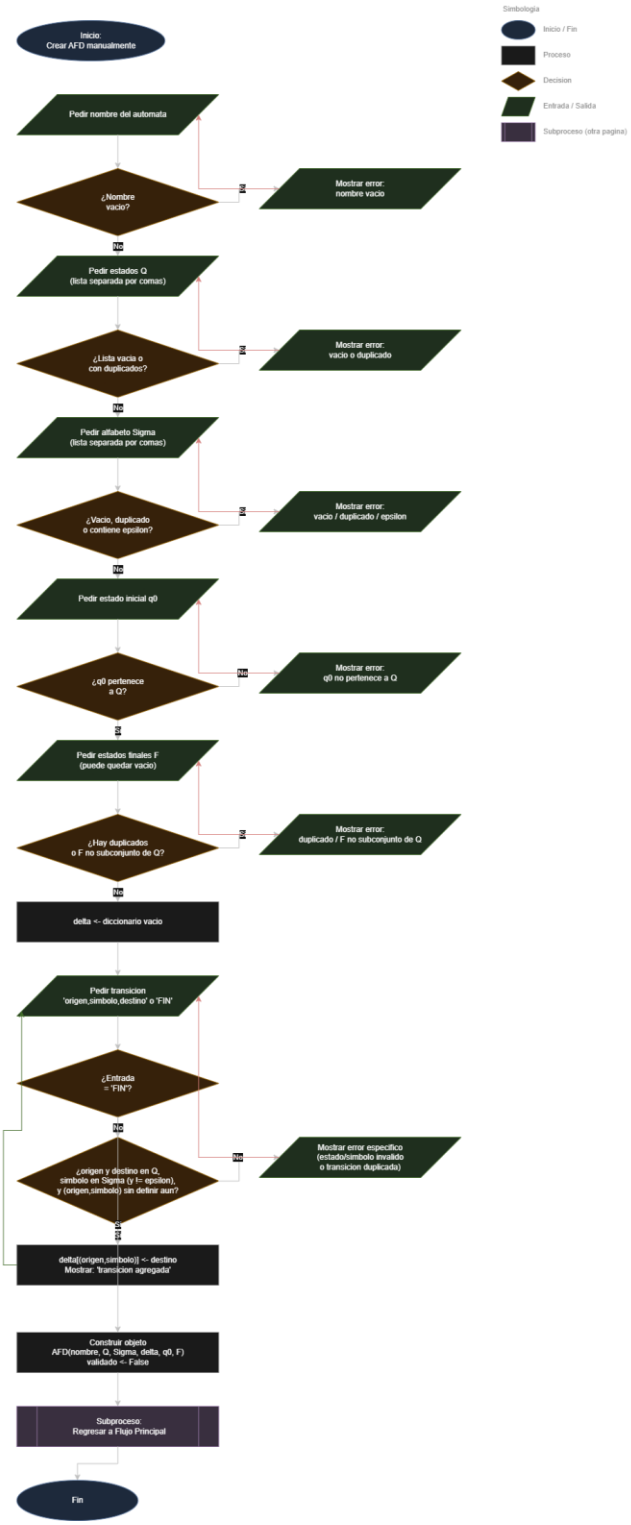
Los diagramas se elaboraron en draw.io; los archivos editables están en `Documentacion/diagramas`.

11.1 Diagrama general



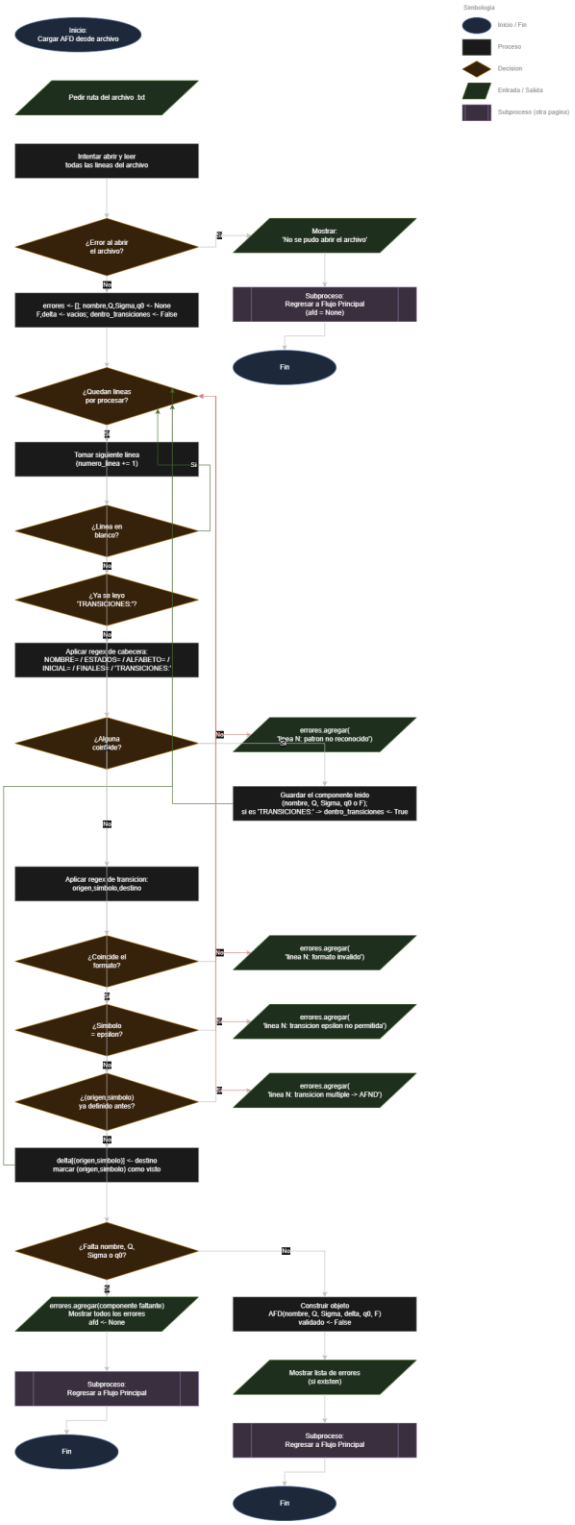
Flujo principal del programa

11.2 Opción 1



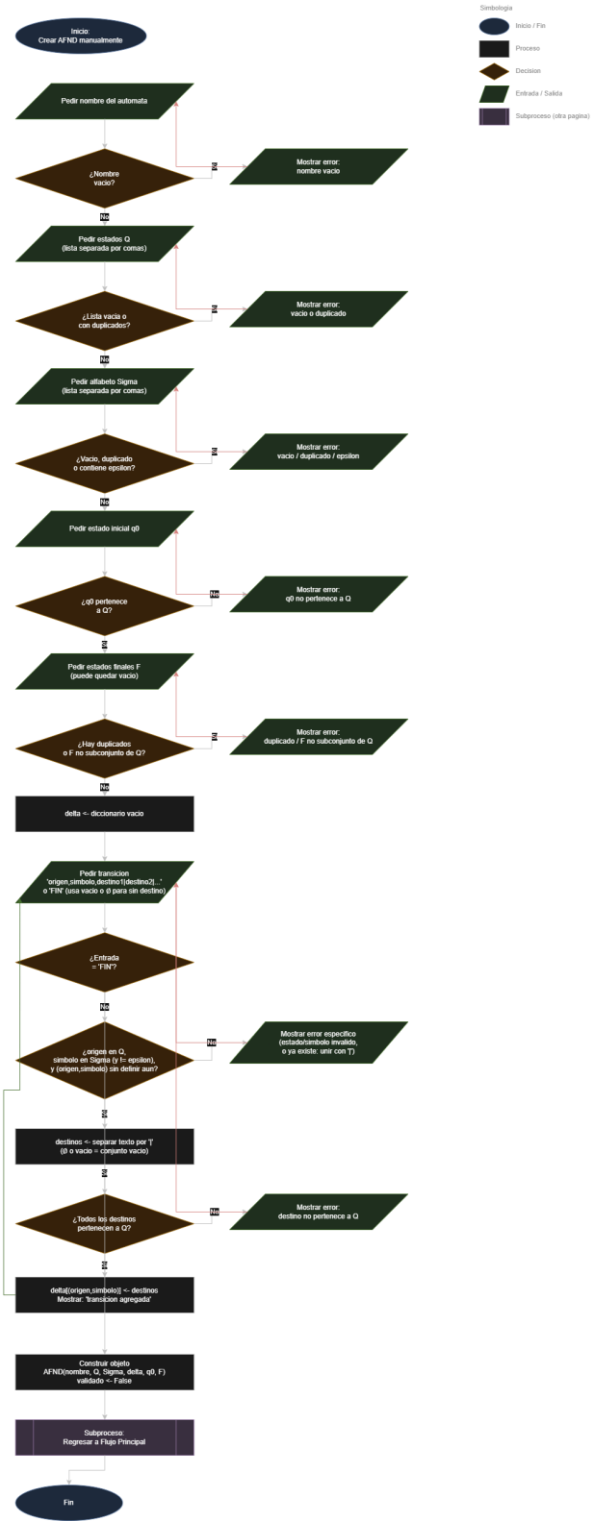
Crear un AFD manualmente

11.3 Opción 2



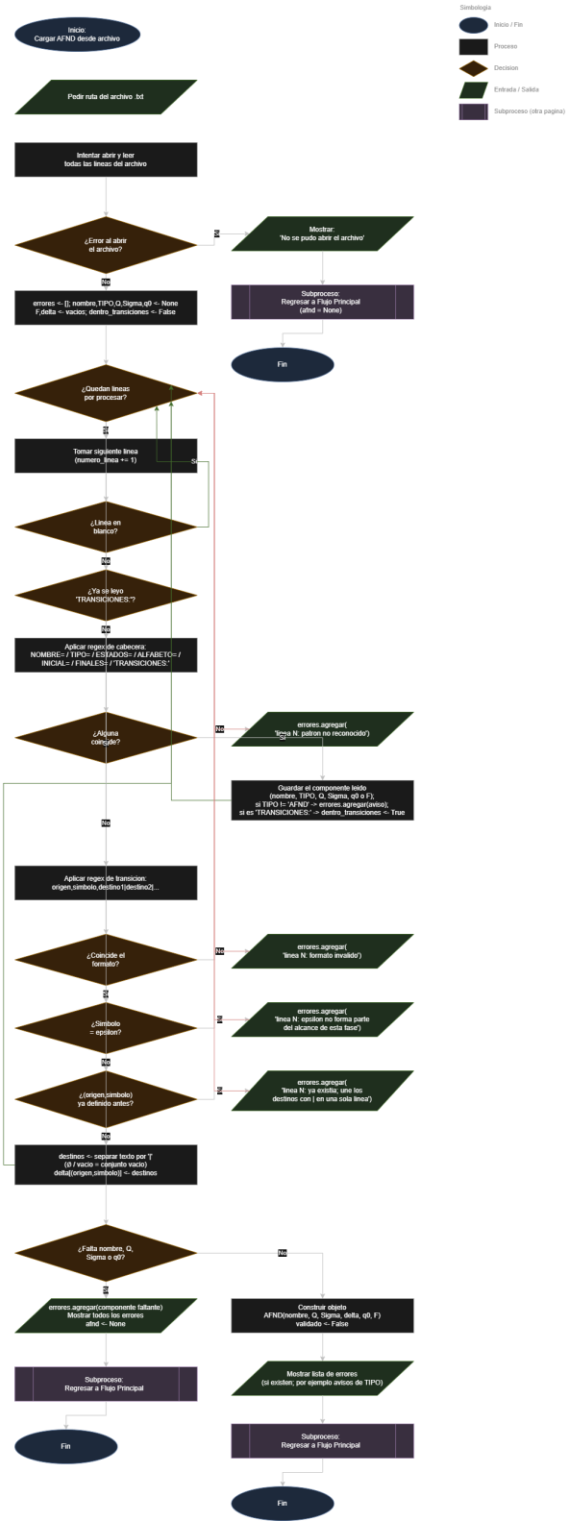
Cargar un AFD desde archivo

11.4 Opción 3



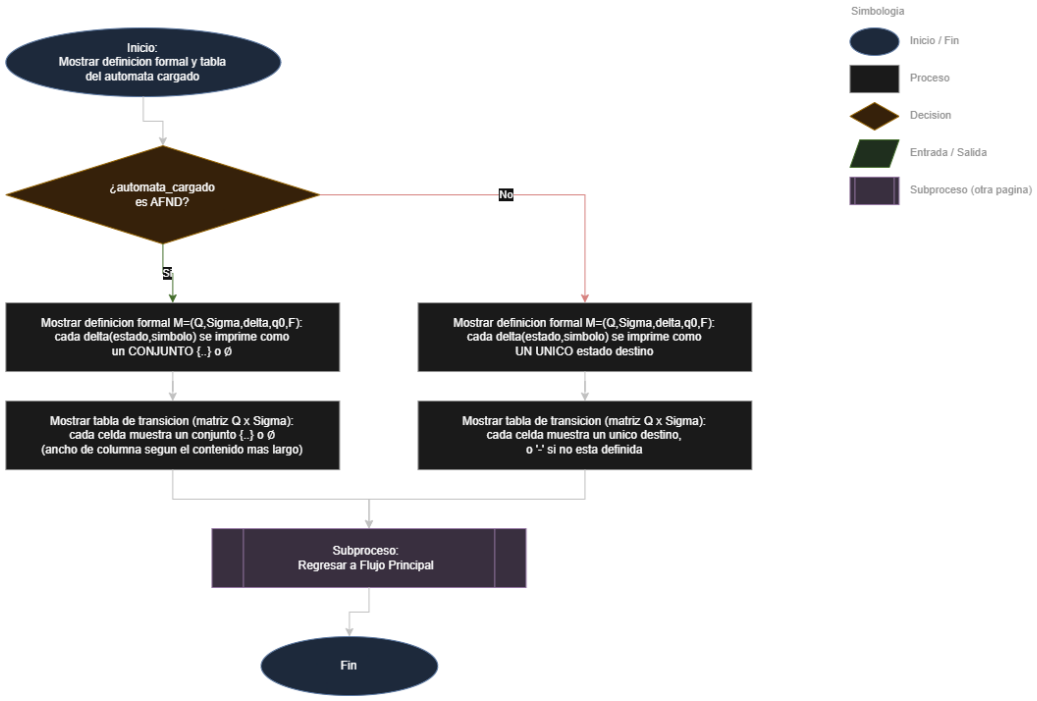
Crear un AFND manualmente

11.5 Opción 4



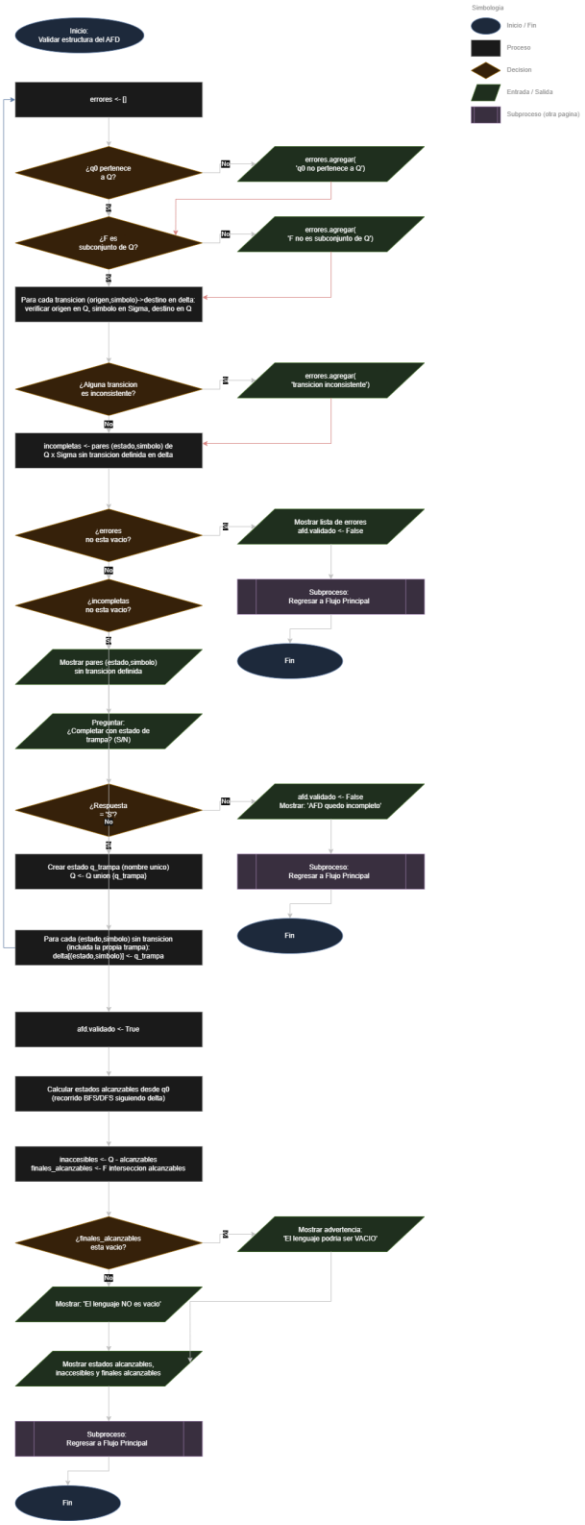
Cargar un AFND desde archivo

11.6 Opción 5



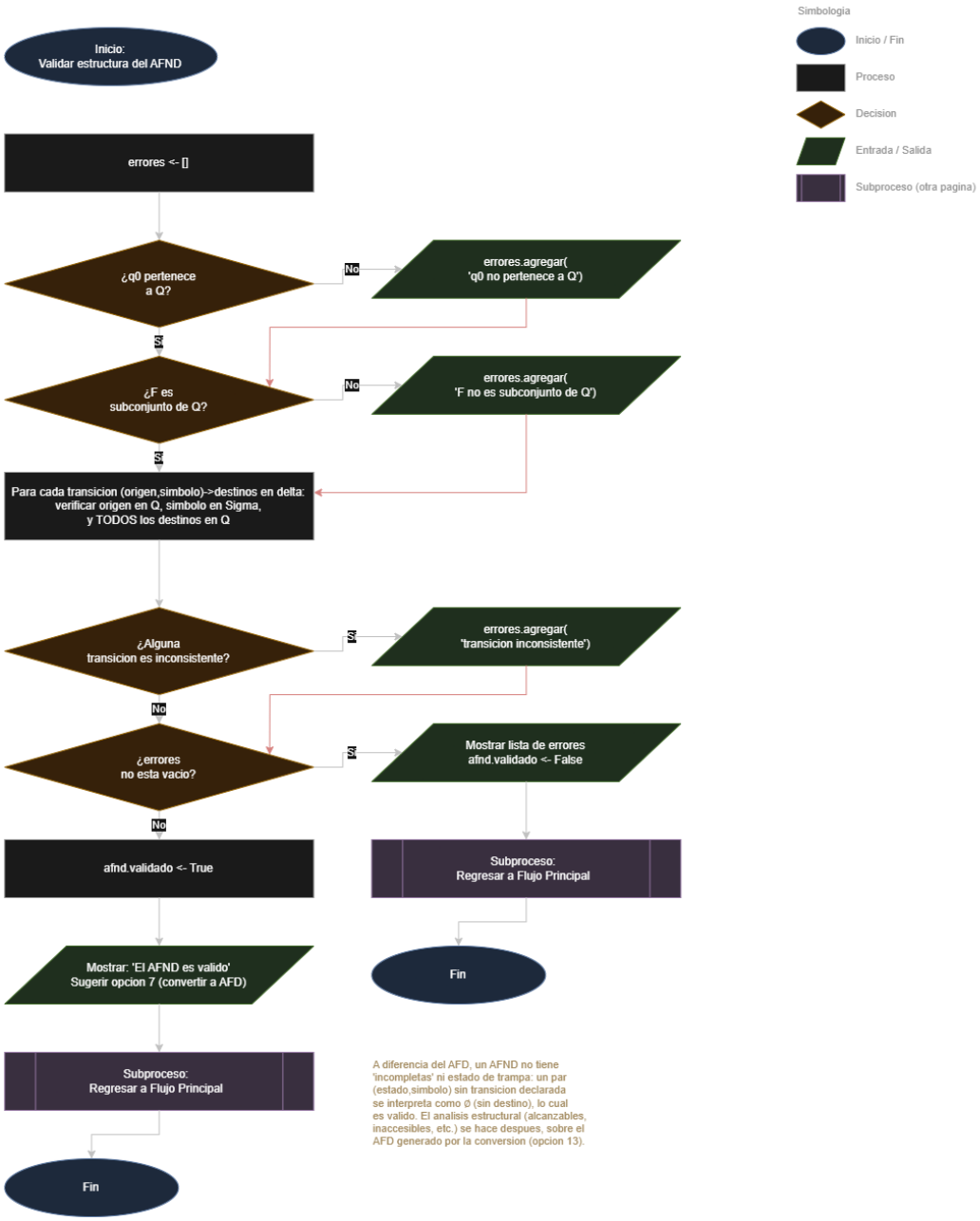
Mostrar la definición formal y la tabla

11.7 Opción 6 (AFD)



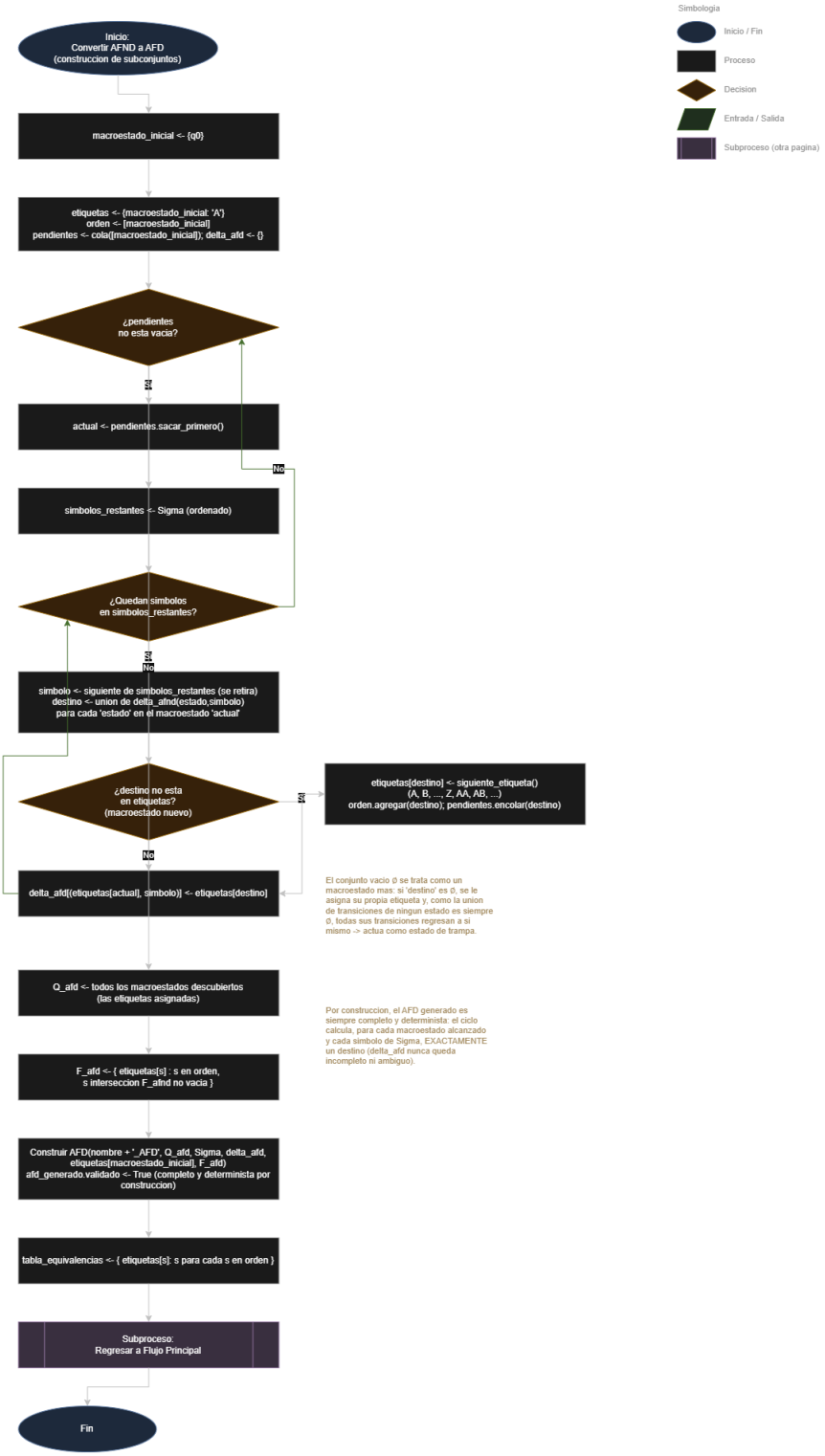
Validar la estructura de un AFD

11.8 Opción 6 (AFND)



Validar la estructura de un AFND

11.9 Opción 7



Convertir el AFND en un AFD equivalente

11.10 Opción 8

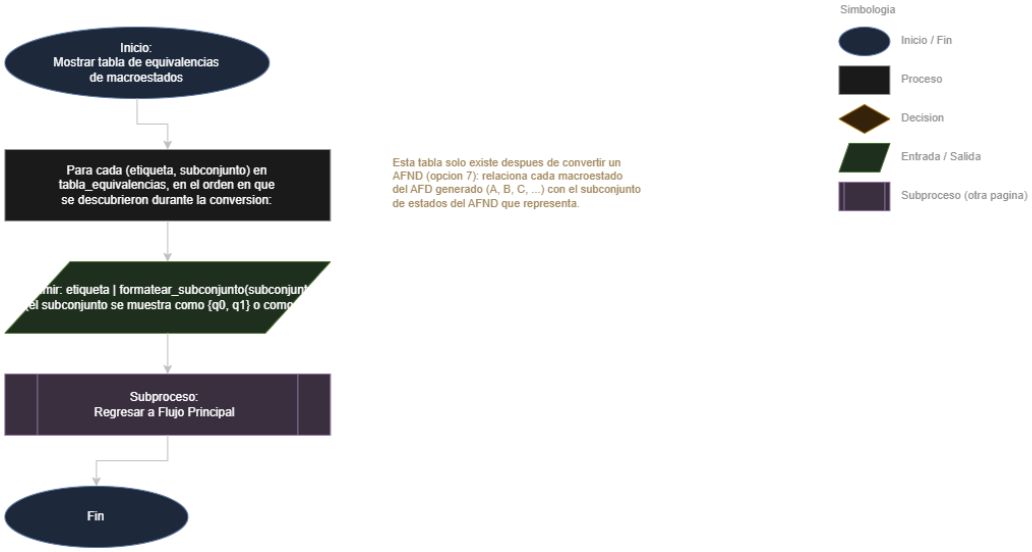
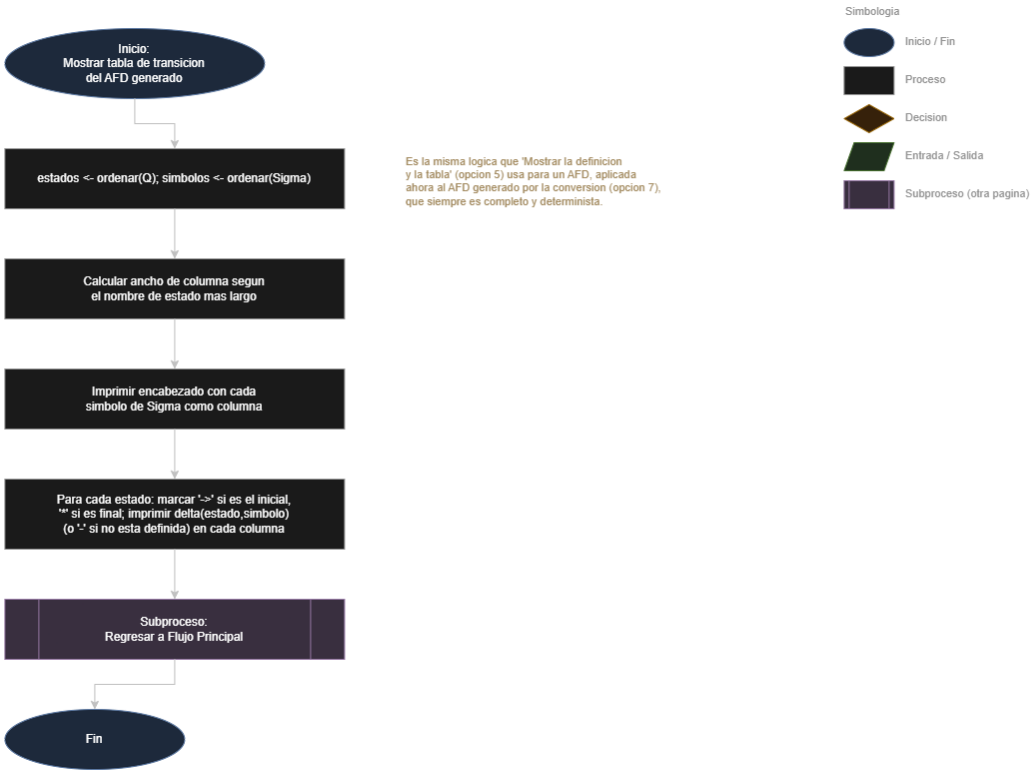


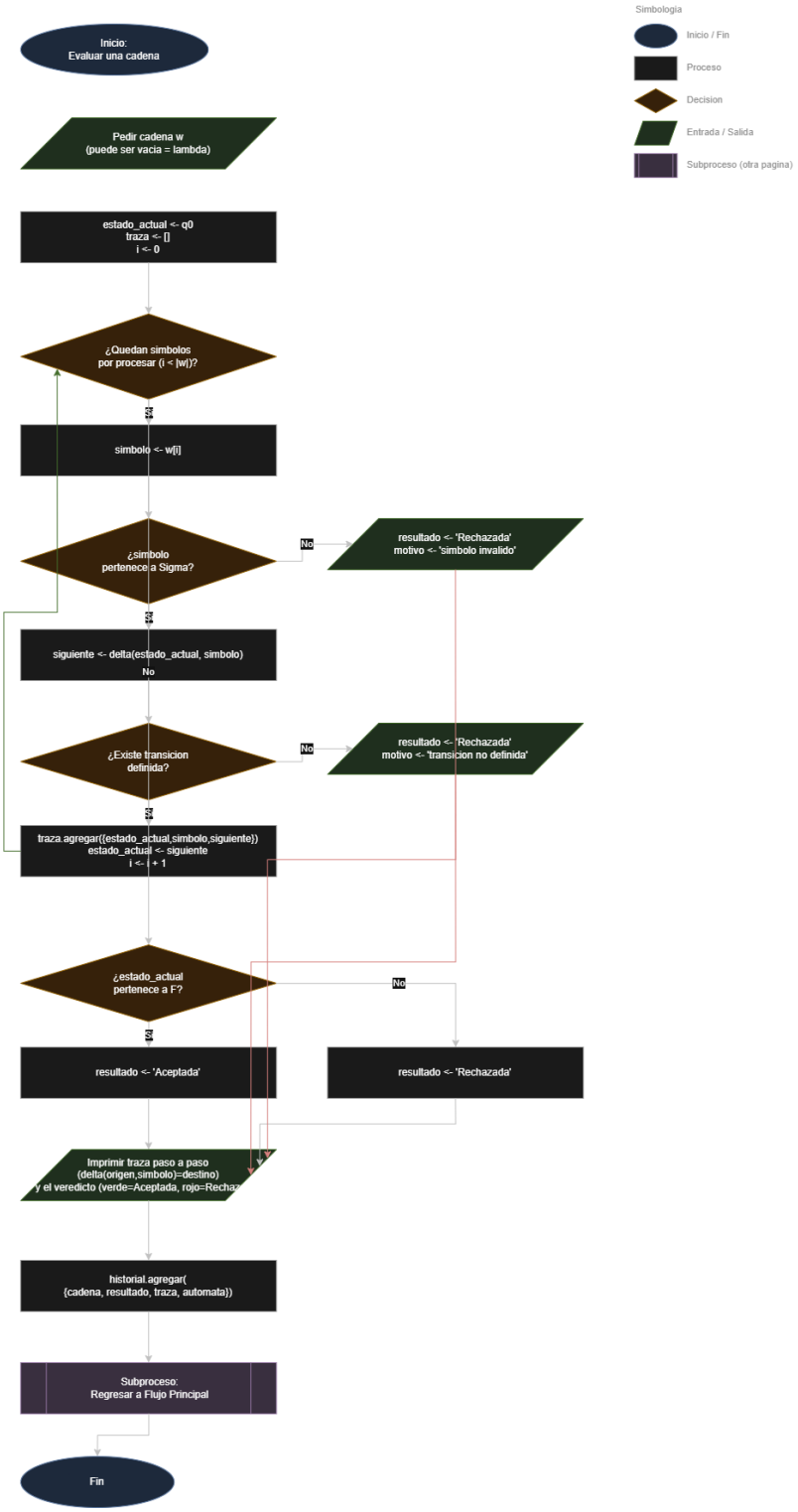
Tabla de equivalencias de macroestados

11.11 Opción 9



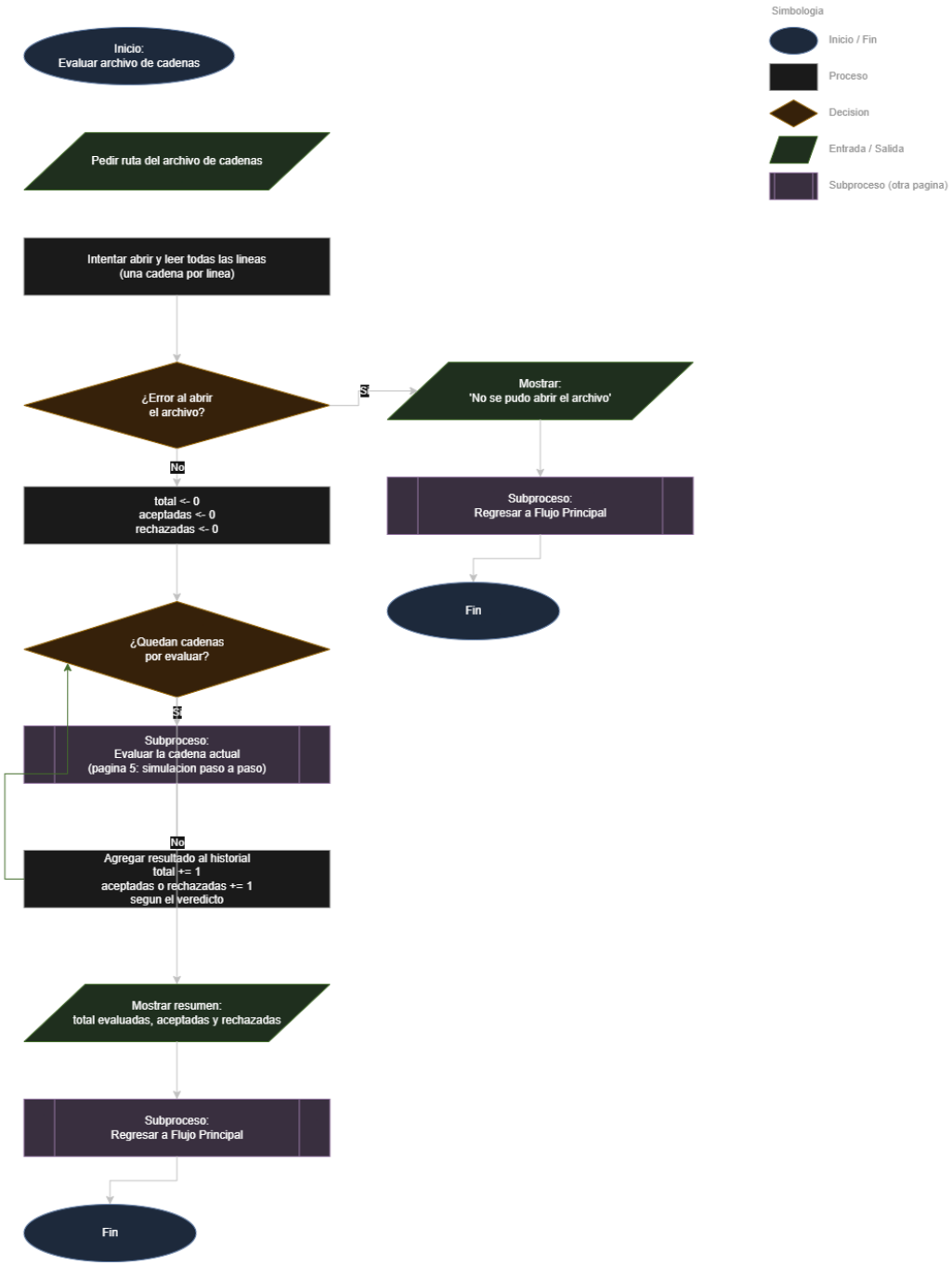
Mostrar el AFD generado

11.12 Opción 10



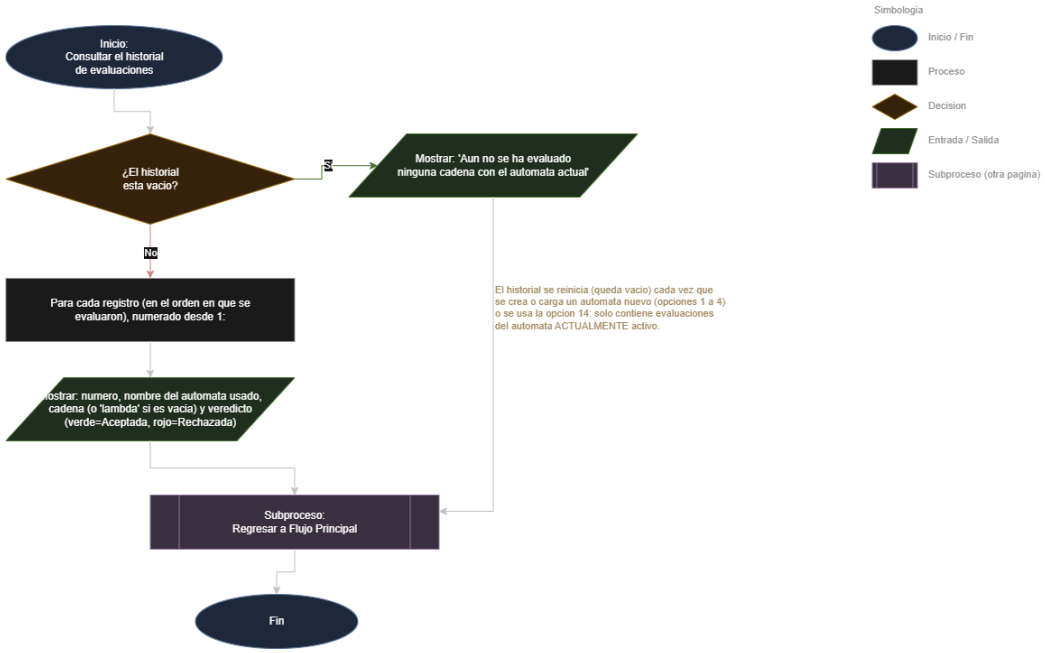
Evaluar una cadena

11.13 Opción 11



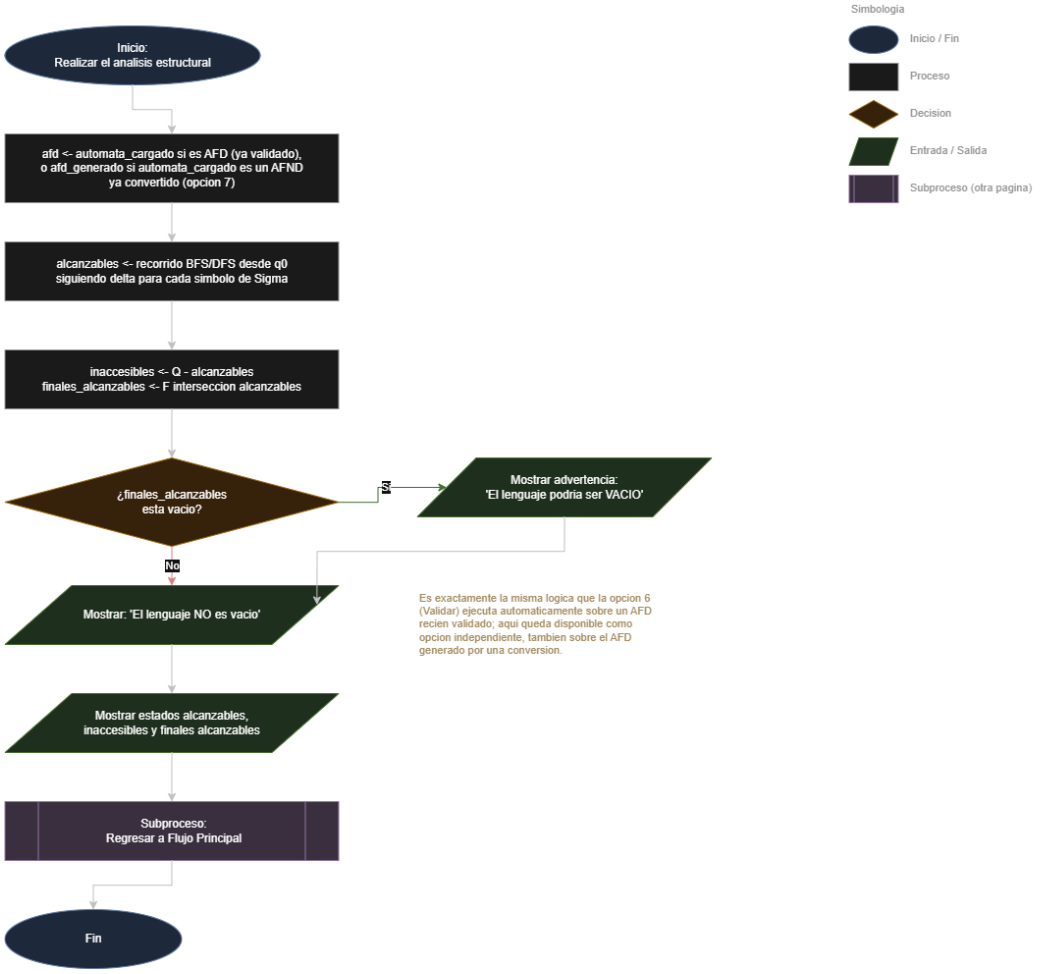
Evaluar un archivo de cadenas

11.14 Opción 12



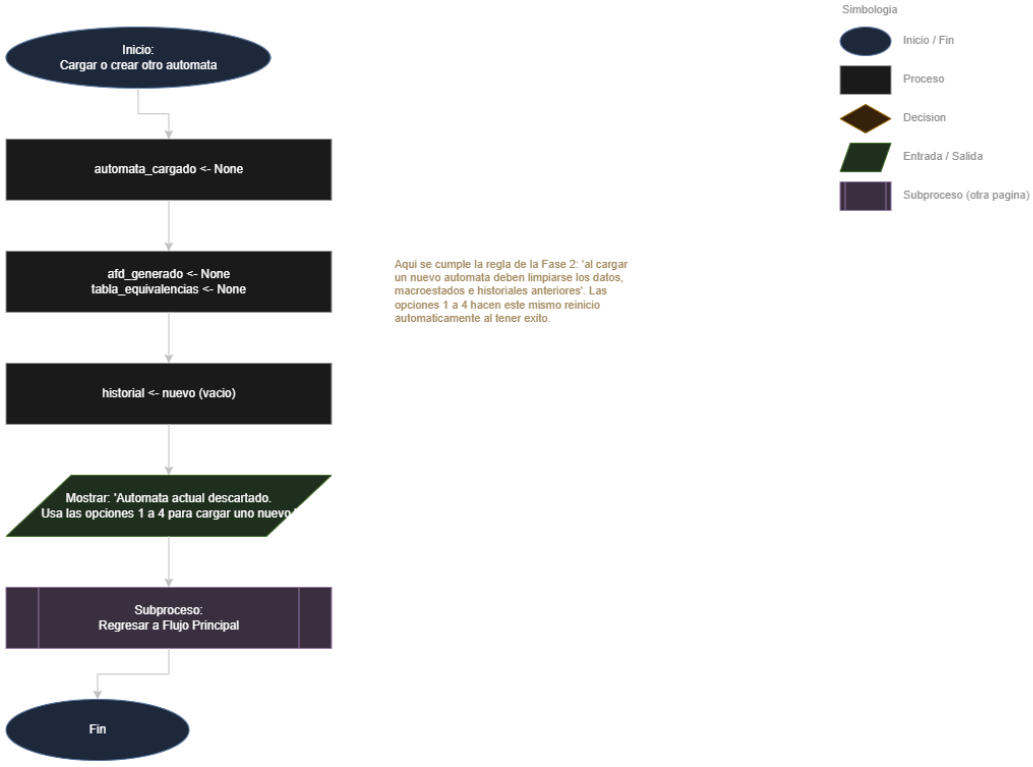
Consultar el historial

11.15 Opción 13



Análisis estructural

11.16 Opción 14



Cargar o crear otro autómata

11.17 Opción 15



Salir

12. Casos de prueba

El detalle completo, con pasos, resultado esperado y resultado obtenido, está en el Documento de análisis y diseño. Resumen de los casos obligatorios:

Caso obligatorio	Archivo	Resultado
AFD válido y completo	afd_ab.txt	Válido; aab y ab aceptadas
AFD incompleto y estado de trampa	afd_incompleto.txt	4 transiciones faltantes; se agrega q_trampa
AFND con dos destinos para un par	afnd_dos_destinos.txt	δ(q0,a) = {q0, q1}; 4 macroestados
Al menos tres macroestados	afnd_tres_macroestados.txt	4 macroestados
Conversión que alcanza ∅	afnd_vacio.txt	B = ∅ como estado de trampa
Macroestado final mixto	afnd_macroestado_mixto.txt	B = {m0, m1} final
Cadena aceptada, rechazada, λ y fuera de Σ	cadenas.txt, cadenas_afnd.txt	Veredictos con traza y motivo
Errores sintácticos	afnd_errores_sintacticos.txt	6 errores con número de línea; no válido
Estados o símbolos inexistentes	afnd_estados_inexistentes.txt	3 errores; conversión bloqueada
Comparación AFND vs AFD	cadenas_afnd.txt (opción 11)	7/7 cadenas coinciden